

Agenți Inteligenți pentru Jocul Snake cu Obstacole folosind Reinforcement Learning

Damian Alexandru

Horneț Alex Andrei

Rizea Mihai Marius

Zamfir Cezara Maria

Proiect IRL 2025-2026

Cuprins

1	Alegerea Temei & Formularea Problemei	2
1.1	Motivația alegerii temei	2
1.2	Problema ca MDP	2
2	Environment: Implementare și Design	3
2.1	Implementare și Obstacole Fixe	3
2.2	Curriculum Learning	3
3	Algoritmi Implementați: Teorie și Practică	4
3.1	Fundamente: Q-Learning și Double Q-Learning	4
3.1.1	Teoria Q-Learning	4
3.1.2	Agent 1: Double Q-Learning (Tabular)	4
3.2	Deep Reinforcement Learning (DQN)	4
3.2.1	De la Tabele la Rețele Neurale	4
3.2.2	Agent 2: Arhitectura DQN Implementată	5
3.3	Agent 3: Dueling DQN	5
3.4	Agent 4: Proximal Policy Optimization (PPO)	5
4	Experimente & Calibrare	6
5	Rezultate Finale & Benchmark	7
5.1	Analiză Comparativă	7
5.2	Interpretarea Rezultatelor	7
6	Concluzii	8

1 Alegerea Temei & Formularea Problemei

1.1 Motivația alegerii temei

Am ales jocul Snake cu obstacole deoarece prezintă caracteristici importante pentru învățarea prin reinforcement:

- **Delayed reward:** Feedback-ul pozitiv apare doar la consumarea hranei.
- **Planning:** Creșterea progresivă a șarpelui necesită planificare pe termen lung.
- **Exploration:** Agentul trebuie să exploreze activ spațiul pentru a localiza resurse.
- **Navigare complexă:** Evitarea simultană a obstacolelor fixe și a propriului corp.

1.2 Problema ca MDP

Am definit problema ca un Markov Decision Process (MDP):

Starea (State): 11 valori binare despre ce vede șarpele:

- 3 valori: Pericol în față / la dreapta / la stânga?
- 4 valori: În ce direcție mă misc acum? (sus/jos/stânga/dreapta)
- 4 valori: Unde e mâncarea? (stânga/dreapta/sus/jos față de mine)

Acțiunile (Actions): 3 opțiuni simple (Mergi înainte, Întoarce-te la dreapta, Întoarce-te la stânga).

Recompense (Rewards):

- +10: Am mâncat.
- -10: M-am lovit (zid, obstacol, sau propriul corp).
- +0.1 / -0.1: Reward shaping pentru apropiere/îndepărtare de mâncare.

2 Environment: Implementare și Design

2.1 Implementare și Obstacole Fixe

Am creat un environment personalizat în `game.py`. O modificare majoră față de Snake-ul clasic este introducerea **obstacolelor fixe** pentru a asigura reproductibilitatea experimentelor.

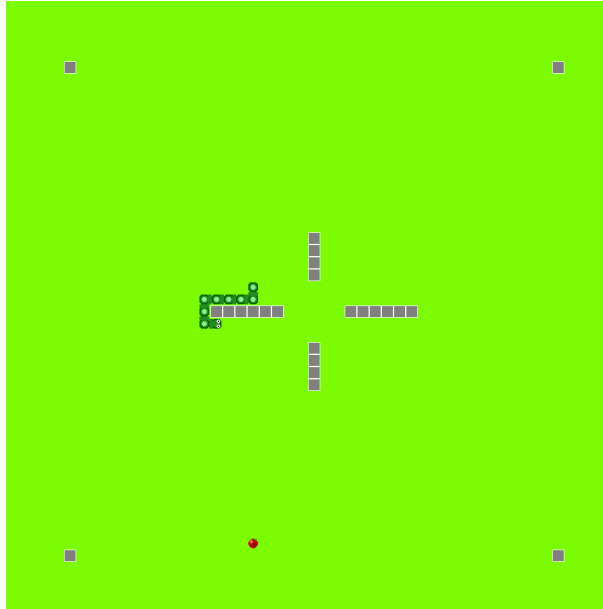


Figura 1: Captură din joc (Environment) rulând în mod grafic.

2.2 Curriculum Learning

Similar procesului educațional uman, antrenamentul începe cu task-uri simple, progresând gradual spre complexitate maximă. Harta crește de la 200x200 px (Small) până la 1000x1000 px (XXL).

3 Algoritmi Implementați: Teorie și Practică

Această secțiune detaliază algoritmi utilizați, integrând conceptele teoretice fundamentale conform documentației de proiect.

3.1 Fundamente: Q-Learning și Double Q-Learning

3.1.1 Teoria Q-Learning

Q-Learning este un algoritm **Off-Policy** și **Model-Free** creat de Chris Watkins în 1989. Conceptul central este funcția $Q(s, a)$, care răspunde la întrebarea: "*Cât de utilă este acțiunea 'a' în starea 's' pentru obținerea recompenselor viitoare?*".

Valoarea $Q(s, a)$ reprezintă "calitatea" combinației stare-acțiune. Actualizarea valorilor se face folosind **Ecuția Bellman**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)] \quad (1)$$

Componentele formulei sunt:

- α (**Learning Rate**): Controlează cât de mult învățăm din noua experiență. Un $\alpha = 0.1$ înseamnă că reținem 10% informație nouă și 90% veche.
- γ (**Discount Factor**): Controlează importanța viitorului. $\gamma = 0.9$ indică o planificare pe termen lung.
- r : Recompensa imediată.
- $\max Q(s', a')$: Estimarea celei mai bune valori viitoare (partea Off-Policy).

3.1.2 Agent 1: Double Q-Learning (Tabular)

Deși Q-Learning este robust, în practică suferă de **Maximization Bias** (supraestimarea valorilor Q). Am implementat **Double Q-Learning**, care utilizează două tabele (Q_1 și Q_2) pentru a decupla selecția acțiunii de evaluarea acesteia:

- Q_1 alege acțiunea optimă ($\argmax$).
- Q_2 evaluează valoarea acelei acțiuni.

Limitare: Fiind o metodă tabulară, pe dimensiuni mari (1000×1000 px), spațiul de stări devine prea mare pentru a fi explorat eficient.

3.2 Deep Reinforcement Learning (DQN)

3.2.1 De la Tabele la Rețele Neurale

Metodele tabulare nu scalează. Pentru medii complexe (ex. Atari, Go), numărul de stări este astronomic. Soluția este **Deep Q-Network (DQN)**, unde o rețea neurală aproximează funcția Q :

$$NeuralNetwork(stare) \rightarrow [Q - values \text{ pentru toate acțiunile}]$$

Aceasta permite **generalizarea**: rețeaua învață pattern-uri (ex: "evită zidul"), nu memorează stări individuale.

3.2.2 Agent 2: Arhitectura DQN Implementată

Pentru stabilitate, am implementat componentele standard DQN:

1. Experience Replay Buffer:

- Stochează tranziții $(s, a, r, s', done)$.
- **Scop:** Rupe corelația temporală dintre experiențele consecutive și permite re-folosirea datelor (sample efficiency).

2. Target Network:

- O copie a rețelei principale care se actualizează periodic.
- **Scop:** Rezolvă problema "Moving Target". Fără ea, rețeaua ar încerca să prezică o țintă care se schimbă la fiecare pas, ducând la instabilitate.

3.3 Agent 3: Dueling DQN

Am implementat o arhitectură avansată care descompune Q-value în două fluxuri:

- **Value Stream** $V(s)$: Cât de bună este starea curentă.
- **Advantage Stream** $A(s, a)$: Cât de bună este o acțiune față de celelalte.

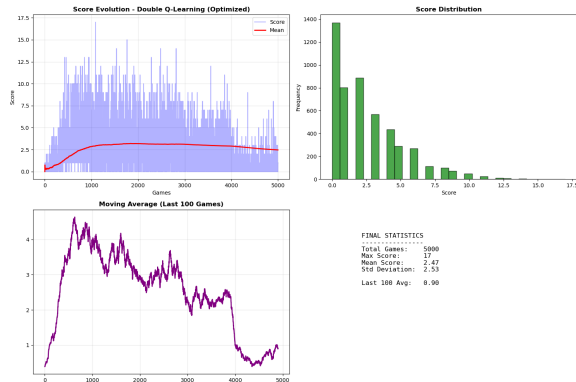
$$Q(s, a) = V(s) + (A(s, a) - \text{mean}(A(s, a)))$$

3.4 Agent 4: Proximal Policy Optimization (PPO)

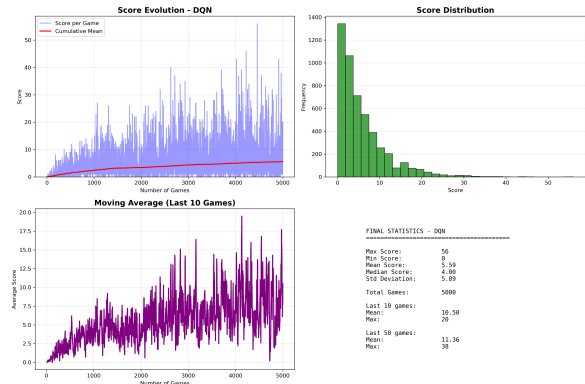
Spre deosebire de DQN (Value-Based), PPO este o metodă **Policy-Based** (Actor-Critic). Acesta optimizează direct politica de acțiune și folosește un mecanism de *clipping* pentru a preveni actualizările drastice care ar putea destabiliza antrenamentul. Este considerat algoritmul "state-of-the-art" pentru stabilitate.

4 Experimente & Calibrare

Am rulat experimente extensive pentru calibrarea hiperparametrilor.

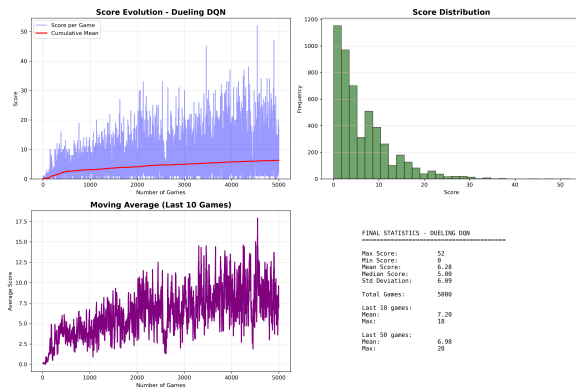


(a) Double Q-Learning: Plafonare rapidă la scor mic.

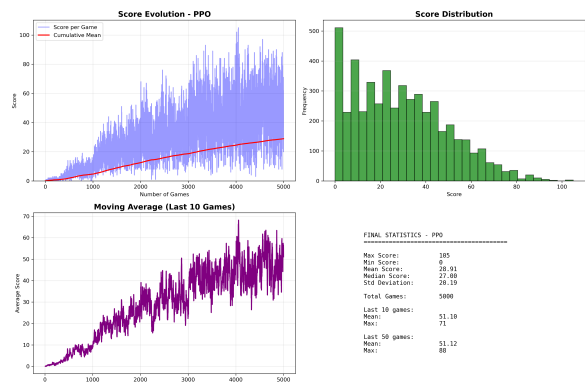


(b) DQN: Învățare vizibilă, dar zgomotoasă.

Figura 2: Curbe de antrenare pentru metodele Q-Learning.



(a) Dueling DQN: Performanță similară cu DQN.



(b) PPO: Creștere stabilă și continuă.

Figura 3: Comparație între metodele Deep RL.

Observații Hiperparametri:

- **Epsilon Decay:** Critic pentru DQN. O scădere prea rapidă duce la convergență prematură.
- **Batch Size:** 1000 s-a dovedit optim. Valori mai mici (ex: 64) duceau la antrenament instabil.
- **Target Update:** Actualizarea Target Network prea frecventă a destabilizat DQN-ul.

5 Rezultate Finale & Benchmark

5.1 Analiză Comparativă

Graficul de mai jos sintetizează performanța finală a celor 4 agenți după 5000 de jocuri.

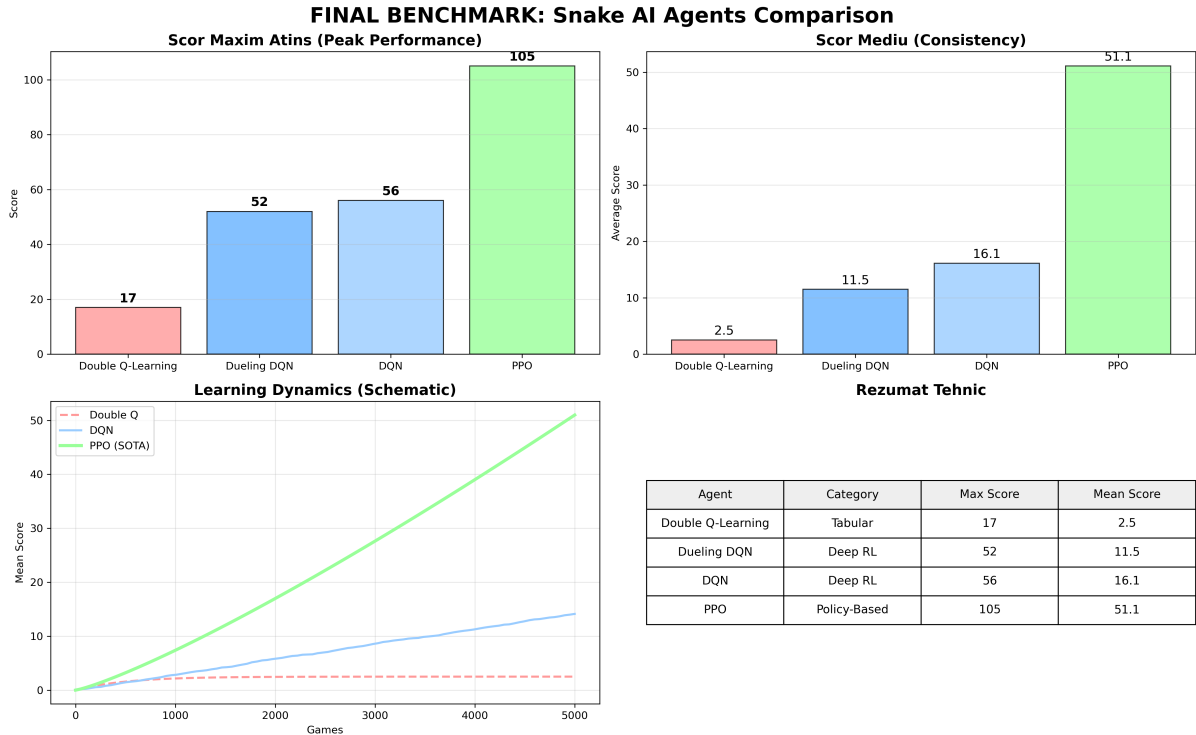


Figura 4: Benchmark Final: Comparație directă a scorurilor maxime și medii.

Agent	Scor Maxim	Scor Mediu	Concluzie
Double Q-Learning	17	2.5	Eșec pe hărți mari (limitare tabulară).
DQN	56	16.1	Bun, dar instabil.
Dueling DQN	52	11.5	Similar cu DQN.
PPO	105	51.1	Superioritate netă.

5.2 Interpretarea Rezultatelor

- Eșecul Tabularului:** Double Q-Learning a demonstrat că metodele fără aproximație (rețele neurale) nu pot gestiona spațiul de stări al jocului Snake pe hărți mari. Agentul a memorat doar situații locale.
- Puterea Generalizării:** DQN și Dueling DQN au reușit să "înțeleagă" conceptul de obstacol și mâncare, obținând scoruri de peste 3 ori mai mari decât metoda tabulară.
- Supremația PPO:** Algoritmul Policy-Based a demonstrat cea mai bună stabilitate și a atins un scor maxim de 105, dublu față de metodele Value-Based. Acesta a navigat cel mai natural printre obstacole.

6 Concluzii

Proiectul a demonstrat evoluția algoritmilor de Reinforcement Learning. Trecerea de la Q-Tables la Deep Q-Networks este esențială pentru medii complexe. Introducerea conceptelor precum *Experience Replay* și *Target Networks* (din teoria DQN) sunt vitale pentru stabilitate. Totuși, algoritmi moderni de tip Policy Gradient (PPO) oferă rezultate superioare în probleme de control continuu și navigare.

Hiperparametru	Valoare	Rol
Gamma (γ)	0.99	<i>Focus Termen Lung</i> (Evită lăcomia imediată)
Clip Range (ϵ)	0.2	<i>Stabilitate</i> (Previne schimbările haotice)
Entropy Coeff	0.01	<i>Explorare</i> (Previne blocarea în bucle)
Learning Rate (α)	3×10^{-4}	<i>Pasul de Învățare</i> (Cât de fin se ajustează rețeaua)
Update Horizon	2000	<i>Memorie Batch</i> (Analiză traiectorie)
Hidden Layers	2 straturi $\times$ 256	<i>Inteligență</i> (Puterea de a procesa combinații complexe)

Tabela 1: Hiperparametrii utilizați pentru antrenarea agentului PPO